

Program Drill

Four full-page programs, each with the reading sequence to follow, the card run block by block in that order, findings, and verdict
October 2026

How to run it.

A full program is read in a fixed order, not top to bottom. The order below is the same for every program; each program section then lists its blocks in that order with the card's notes for each. Do the program yourself first: write the sequence, run the card per block, write the notes, write findings and verdict. Then uncover. Budget 35 minutes per program for A–C, 25 for D.

The reading order for any program

Step	What to read	Why first	Card emphasis
1	the requirement, rewritten as numbered clauses	everything is judged against it	none yet
2	imports and types (<code>import</code> , <code>interface</code> , <code>type</code> , <code>enum</code>)	they tell you the shape of every value before you meet it; note every <code>?</code> , every <code> </code> , <code>null</code> , every <code>any</code>	Q3, Q5: which fields can be nothing
3	module-level state (<code>const x = new Map()</code> , <code>let count = 0</code> , <code>const cache = {}</code> outside any function)	shared by every call; this is where races and leaks live	Q6: who else; Q7: who removes
4	the exported entry points (<code>export function</code> , <code>app.get(...)</code> , <code>export class</code>), signatures only	the contract: parameters, return type, <code>async</code> or not	Q4: ticket?; Q1: declared return
5	each entry point's body , in the order they are called in real use	this is the work; run the card line by line	all seven
6	helpers , when the entry point calls them	read a helper at the moment it is called, with the caller's argument values in hand	Q4 and Q6 on the boundary: does the helper mutate what it was given, return a ticket, return an alias
7	cleanup and lifecycle (<code>setInterval</code> , <code>on(...)</code> , <code>process.on</code> , timers)	what runs without being called, and what never stops	Q6, Q7: cleared? unref'd? removed?
8	back to the clauses: point each clause at a line	the verdict	Review Q1, Q16

Say the order out loud before each program: *types, state, signatures, bodies, helpers, lifecycle, clauses*.

Program A — Order totals service

Requirement

1. `GET /orders/:userId/total` returns `{ totalCents, count }` for the user's paid orders.
2. Amounts are stored in dollars as floats; return integer cents.
3. Results are cached per user for 30 s; a cache hit must not call the database.
4. Only the authenticated user may read their own total (user id comes from the verified token).
5. Unknown user: 404. Missing or invalid token: 401.
6. Do not modify the arrays returned by the database.

```

1  import { Request, Response, NextFunction } from "express";
2  import { db } from "../db";
3  import { verify } from "../auth";
4
5  interface Order {
6    id: number;
7    userId: string;
8    status: "paid" | "pending" | "refunded";
9    amount: number; // dollars
10   discount?: number; // dollars
11 }
12
13 interface Totals { totalCents: number; count: number }
14
15 const CACHE_TTL = 30_000;
16 const cache: Record<string, { at: number; value: Totals }> = {};
17
18 function sumOrders(orders: Order[]): Totals {
19   const paid = orders.filter((o) => o.status == "paid");
20   paid.sort((a, b) => a.amount - b.amount);
21   let total = 0;
22   for (const o of paid) {
23     if (o.discount) o.amount = o.amount - o.discount;
24     total += o.amount * 100;
25   }
26   return { totalCents: total, count: paid.length };
27 }
28
29 async function loadOrders(userId: string): Promise<Order[]> {
30   const rows = await db.query("SELECT * FROM orders WHERE user_id = $1", [userId]);
31   return rows;
32 }
33
34 export async function getTotal(req: Request, res: Response, next: NextFunction) {
35   const token = req.headers.authorization?.split(" ")[1];
36   const claims = verify(token!);
37   if (!claims) return res.status(401).json({ error: "unauthorized" });
38
39   const userId = req.params.userId;
40   const hit = cache[userId];
41   if (hit && Date.now() - hit.at < CACHE_TTL) {
42     return res.json(hit.value);
43   }
44
45   const orders = loadOrders(userId);
46   if (orders.length === 0) return res.status(404).json({ error: "no such user" });
47
48   const totals = sumOrders(await orders);
49   cache[userId] = { at: Date.now(), value: totals };
50   res.json(totals);
51 }
52
53 setInterval(() => {
54   const now = Date.now();
55   for (const k in cache) {
56     if (now - cache[k].at > CACHE_TTL) delete cache[k];
57   }
58 }, 60_000);

```

Reading sequence for A

Step	Lines	Block
2 types	5–13	Order, Totals
3 state	15–16	CACHE_TTL, cache
4 signature	34	getTotal
5 body	35–51	getTotal
6 helper	29–32	loadOrders (called at 45)

Step	Lines	Block
6 helper	18–27	<code>sumOrders</code> (called at 48)
7 lifecycle	53–58	sweep interval
8 clauses		

Step 2: types (5–13)

Line	Note
8	<code>status</code> is a three-way literal union; any comparison must use one of those exact strings
9	<code>amount</code> is dollars, a float; clause 2 means every total must be rounded to integer cents
10	<code>discount?</code> can be absent: Q5 <code>yes</code> on every <code>o.discount</code> read
13	<code>Totals</code> is what the cache stores and what the handler returns

Step 3: module state (15–16)

Line	Note
16	<code>cache</code> is a plain object keyed by <code>userId</code> from the URL: user-controlled keys on a prototype-bearing object (Q6: shared by every request; Q7: sweep at 53 removes; <code>"constructor"</code> as a <code>userId</code> would hit an inherited key)

Step 4: signature (34)

Note
<code>async</code> , so it returns a Promise; Express 4 ignores it, so any throw inside hangs the request unless caught; declared no return type (fine for a handler)

Step 5: body of `getTotal` (35–51)

Line	Note
35	<code>token</code> holds a string or <code>undefined</code> (no header, or header without a space): Q5 yes
36	<code>token!</code> claims non-null; <code>verify(undefined)</code> is called with the claim unverified; depends on <code>verify</code> tolerating it; the <code>!</code> is a hole
37	returns after the 401; correct
39	<code>userId</code> holds the URL param, a string, caller-controlled
40	<code>hit</code> holds the cache entry or <code>undefined</code>
41	TTL check: <code><</code> so exactly 30 s is stale; fine; <code>return</code> present
45	<code>orders</code> holds a PROMISE: <code>loadOrders</code> is <code>async</code> and there is no <code>await</code>
46	<code>orders.length</code> on a Promise is <code>undefined</code> ; <code>undefined === 0</code> is false; the 404 branch never fires
48	<code>await orders</code> here unwraps it, so <code>sumOrders</code> receives the array; the 404 check on 46 happened on the wrong thing
49	writes the cache; between 41 and 49 another request for the same user misses too (minor race; two DB calls, same result)
50	no <code>return</code> , but it is the last statement; fine
35–50	clause 4: <code>userId</code> comes from <code>req.params</code> (line 39), never compared to <code>claims</code> : any authenticated user can read any user's total

Step 6: helper `loadOrders` (29–32), called at 45 with `userId`

Line	Note
30	parameterized query; awaited; <code>rows</code> is whatever the driver returns, typed <code>any</code>
31	returns <code>rows</code> as <code>Order[]</code> with no check; an empty result is <code>[]</code> , so <code>length === 0</code> is the right test if it had been awaited

Step 6: helper `sumOrders` (18–27), called at 48 with the awaited array

Line	Note
19	<code>==</code> instead of <code>===</code> ; works for strings but flag; <code>paid</code> holds a NEW array whose elements are the DB's objects (shared)
20	<code>sort</code> mutates <code>paid</code> , which is the new array, so the DB array is not reordered: fine; but the sort is unnecessary
23	<code>if (o.discount)</code> skips a discount of <code>0</code> (correct by accident); <code>o.amount = ...</code> MUTATES the order object, which is shared with the array from the DB: clause 6 violated; and the mutation is permanent for anyone else holding the row
24	<code>o.amount * 100</code> on floats: <code>19.99 * 100</code> is <code>1998.9999...</code> ; <code>total</code> is not an integer: clause 2 violated
26	returns <code>count: paid.length</code> ; correct

Step 7: lifecycle (53–58)

Line	Note
53	interval never cleared and not <code>.unref()</code> 'd: process cannot exit; the closure holds <code>cache</code>
55	<code>for...in</code> over a plain object: also walks inherited enumerable keys if the prototype was polluted; use <code>Object.keys</code> or a <code>Map</code>
56	<code>delete</code> on a plain object: fine; on a <code>Map</code> it would be wrong

Step 8: clauses

Clause	Line	Status
1 total of paid orders	19–26	implemented, but amounts corrupted by 23 and not integer by 24
2 integer cents	24	NOT implemented: no rounding
3 cache, no DB on hit	40–42	implemented
4 own total only	39	NOT implemented: <code>userId</code> from the URL, never checked against <code>claims</code>
5 404 / 401	46 / 37	404 never fires (45 missing await); 401 fine
6 do not modify DB arrays	23	violated: element mutated

Findings (ranked)

- Line 39 vs 36: authorization bypass; `req.params.userId` is never compared to `claims.sub/claims.userId`; any token reads any user. Use the claim, or 403 on mismatch.
- Line 45: `loadOrders` not awaited; line 46 tests `.length` on a Promise, so the 404 path is dead; `const orders = await loadOrders(userId)`.
- Line 23: `o.amount = o.amount - o.discount` mutates the DB row object; violates clause 6 and corrupts shared data; compute into a local.
- Line 24: float multiplication; `totalCents` is not an integer; `Math.round(o.amount * 100)` per order, or work in cents.
- Line 36: `token!` with no check; `verify` receives `undefined` on a missing header; check before calling.
- Line 16: `cache` keyed by user-controlled strings on a plain object; `"__proto__"/"constructor"` as a `userId`; use a `Map`.
- Line 53: interval never cleared or `unref`'d.
- Line 19: `==`; harmless here, flag.

Positives: parameterized SQL; cache TTL check with `return`; `filter` makes a new array so the sort on 20 does not reorder the DB result; a sweep exists.

Confidence 1 • Reject.

Clause 4 is a security hole; clauses 2, 5, 6 fail.

Program B — Job queue with retry and concurrency limit

Requirement

1. `enqueue(job)` adds a job; `start()` runs jobs with at most `concurrency` running at once.
2. A failing job is retried up to `maxRetries` times with 500 ms between attempts; after that it is moved to `failed`.
3. `stop()` stops picking up new jobs and resolves once running jobs finish.
4. `stats()` returns `{ queued, running, done, failed }` counts.
5. Jobs must run in FIFO order of enqueue.

```
1  type Job = { id: string; run: () => Promise<void>; attempts?: number };
2
3  export class Queue {
4    private queued: Job[] = [];
5    private running = new Set<string>();
6    private done = 0;
7    private failed: Job[] = [];
8    private stopped = false;
9    private timer?: ReturnType<typeof setInterval>;
10
11    constructor(private concurrency = 2, private maxRetries = 3) {}
12
13    enqueue(job: Job) {
14      this.queued.push(job);
15    }
16
17    start() {
18      this.timer = setInterval(() => this.tick(), 50);
19    }
20
21    async stop() {
22      this.stopped = true;
23      clearInterval(this.timer);
24      while (this.running.size > 0) await sleep(10);
25    }
26
27    stats() {
28      return { queued: this.queued.length, running: this.running.size, done: this.done, failed:
this.failed.length };
29    }
30
31    private tick() {
32      if (this.stopped) return;
33      while (this.running.size < this.concurrency && this.queued.length) {
34        const job = this.queued.pop()!;
35        this.running.add(job.id);
36        this.execute(job);
37      }
38    }
39
40    private async execute(job: Job) {
41      try {
42        await job.run();
43        this.done++;
44      } catch (e) {
45        job.attempts = (job.attempts ?? 0) + 1;
46        if (job.attempts < this.maxRetries) {
47          setTimeout(() => this.queued.push(job), 500);
48        } else {
49          this.failed.push(job);
50        }
51      }
52      this.running.delete(job.id);
53    }
54  }
55
56  const sleep = (ms: number) => new Promise((r) => setTimeout(r, ms));
```

Reading sequence for B

Step	Lines	Block
2 types	1	<code>Job</code>
3 state	4–9	class fields (per instance, shared by every method)
4 signatures	11, 13, 17, 21, 27	constructor, <code>enqueue</code> , <code>start</code> , <code>stop</code> , <code>stats</code>
5 bodies	13–29	public methods in call order: <code>enqueue</code> , <code>start</code> , <code>stats</code> , <code>stop</code>
6 helpers	31–38, 40–53	<code>tick</code> (called by the interval), <code>execute</code> (called by <code>tick</code>)
7 lifecycle	17–19, 21–25, 56	interval start/stop, <code>sleep</code>
8 clauses		

Step 2: types (1)

Note
<code>run</code> returns a Promise: every call to it needs <code>await</code> ; <code>attempts?</code> can be absent: Q5 yes on every read

Step 3: state (4–9)

Line	Note
4	<code>queued</code> is an array; FIFO needs removal from the FRONT (<code>shift</code>), insertion at the back (<code>push</code>)
5	<code>running</code> is a <code>Set</code> of ids: duplicates collapse; two jobs with the same id would under-count
7	<code>failed</code> is an array of the job objects themselves: shared with whoever enqueued them
9	<code>timer?</code> can be <code>undefined</code> before <code>start()</code>

Step 4: signatures

Line	Note
11	parameter properties; defaults 2 and 3
17	<code>start()</code> is sync
21	<code>stop()</code> is async: returns a Promise; callers must await it
27	<code>stats()</code> returns a fresh object of numbers: safe to hand out

Step 5: public bodies

Line	Note
14	<code>push</code> at the back: correct for FIFO entry
18	<code>timer</code> holds the interval id; <code>tick</code> runs every 50 ms, forever until <code>stop</code>
22–23	flag set, interval cleared; <code>clearInterval(undefined)</code> is harmless if <code>start</code> was never called
24	polls <code>running.size</code> every 10 ms until zero: resolves once running jobs finish (clause 3); but retries scheduled by line 47 are not "running" (they sit in a <code>setTimeout</code>), so <code>stop</code> resolves while a retry is still pending, and that retry will push into <code>queued</code> after stop; nobody picks it up; the job is lost
28	counts are consistent with the fields; a job waiting in a 500 ms retry timeout is in NONE of the four counts (it was deleted from <code>running</code> at 52 and not yet pushed to <code>queued</code>)

Step 6: `tick` (31–38), called by the interval every 50 ms

Line	Note
33	loop while there is capacity and work
34	<code>pop()</code> removes from the BACK: LIFO, not FIFO; clause 5 violated; the <code>!</code> is fine because <code>length</code> was checked
35	adds id to <code>running</code>
36	<code>this.execute(job)</code> is async and NOT awaited: intentional fire-and-forget so the loop can start several; but a rejection from <code>execute</code> itself (not from <code>job.run</code> , which is caught inside) would be unhandled; <code>execute</code> catches everything from <code>run</code> , so the only escape is a throw inside the <code>catch</code> block (e.g. <code>this.failed.push</code> cannot throw); acceptable, say so

Step 6: `execute` (40–53), called by `tick` with one job

Line	Note
42	<code>await job.run()</code> inside the <code>try</code> : a rejection is caught; correct
43	<code>done++</code> on success
45	<code>attempts</code> incremented; <code>?? 0</code> handles the absent case; this MUTATES the job object the caller enqueued (acceptable for a queue, say so)
46	boundary: <code>attempts < maxRetries</code> with <code>maxRetries = 3</code> : attempts 1 and 2 retry, attempt 3 fails: that is 3 attempts total, 2 retries; the requirement says "retried up to <code>maxRetries</code> times", which would be 4 attempts; off-by-one against the wording
47	<code>setTimeout</code> pushes the job back after 500 ms; the timer is never stored, so <code>stop()</code> cannot cancel it; and <code>push</code> puts the retry at the BACK, while <code>pop</code> takes from the back, so a retried job jumps the queue
49	moves to <code>failed</code>
52	<code>running.delete</code> runs on both paths; correct; but it runs BEFORE the retry timer fires, so during the 500 ms the job is counted nowhere (see stats note)

Step 7: lifecycle

Line	Note
18	interval not <code>.unref()</code> 'd; <code>stop()</code> clears it, so fine if <code>stop</code> is called; a process that forgets <code>stop</code> cannot exit
47	retry timers are not tracked: cannot be cleared on <code>stop</code> ; leak of pending callbacks and a lost job
56	<code>sleep</code> defined AFTER the class; <code>const</code> is not hoisted, but it is only called inside <code>stop()</code> , which runs later, so the reference is fine at call time; say so rather than flag it

Step 8: clauses

Clause	Line	Status
1 concurrency limit	33	implemented
2 retry with delay, then failed	45–49	implemented; count off by one vs wording; retry timer untracked
3 stop waits for running	24	partly: ignores pending retries
4 stats	28	partly: retry-pending jobs are invisible
5 FIFO	34	NOT implemented: <code>pop</code> is LIFO

Findings

- Line 34: `pop()` takes from the back: LIFO; clause 5 fails on every run with more than one job; use `shift()`.
- Line 47: retry re-enqueue via an untracked `setTimeout`; `stop()` cannot cancel it, so a retry fires after stop and the job is lost; and with `pop` the retried job jumps the queue. Track the timers (or an `inFlightRetries` count) and have `stop` wait for them.
- Line 46: `attempts < maxRetries` gives `maxRetries` total attempts, not `maxRetries` retries; clarify or use `<=`.
- Line 28 / 52: a job in its 500 ms retry wait is in no count; `stats` under-reports; keep it in `running` (or a `retrying` set) until re-enqueued.
- Line 5: `Set<string>` of ids assumes unique ids; two jobs with one id under-count running and allow over-concurrency; use a counter or a `Set<Job>`.
- Line 24: busy-polling with `sleep(10)`; works; a resolver pattern would be cleaner (nit).

Positives: `await job.run()` inside `try` so failures are caught; `running.delete` on both paths; `stats` returns a fresh object; `clearInterval` in `stop`; `?? 0` on `attempts`.

Confidence 2 • Request changes.

Concurrency and retry mechanics are right; FIFO fails outright; stop/stats miss the retry window.

Program C — API client with token refresh

Requirement

1. `get(path)` performs an authenticated GET and returns the parsed JSON.
2. If the access token is expired (401), refresh it once using the refresh token, then retry the request once.
3. Concurrent calls during a refresh must share the single refresh, not each refresh.
4. Network errors and non-2xx responses other than 401 throw an `ApiError` with the status.
5. The client must not log tokens.

```
1  class ApiError extends Error {
2    constructor(public status: number, message: string) { super(message); }
3  }
4
5  export class ApiClient {
6    private access: string;
7    private refreshing: Promise<string> | null = null;
8
9    constructor(private baseUrl: string, access: string, private refresh: string) {
10      this.access = access;
11    }
12
13    async get<T>(path: string): Promise<T> {
14      const res = await this.request(path);
15      if (res.status === 401) {
16        this.access = await this.refreshToken();
17        const retry = await this.request(path);
18        return retry.json();
19      }
20      if (!res.ok) throw new ApiError(res.status, `GET ${path} failed`);
21      return res.json();
22    }
23
24    private request(path: string) {
25      console.log("GET", path, this.access);
26      return fetch(this.baseUrl + path, { headers: { Authorization: "Bearer " + this.access }
27    });
28
29    private async refreshToken(): Promise<string> {
30      if (this.refreshing) return this.refreshing;
31      this.refreshing = (async () => {
32        const res = await fetch(this.baseUrl + "/token", {
33          method: "POST",
34          body: JSON.stringify({ refresh: this.refresh }),
35        });
36        const data = await res.json();
37        return data.access;
38      })();
39      const token = await this.refreshing;
40      this.refreshing = null;
41      return token;
42    }
43  }
```

Reading sequence for C

Step	Lines	Block
2 types	1–3	<code>ApiError</code>
3 state	6–7	<code>access</code> , <code>refreshing</code> (per instance, shared across concurrent calls)
4 signatures	9, 13, 24, 29	constructor, <code>get</code> , <code>request</code> , <code>refreshToken</code>
5 body	13–22	<code>get</code>
6 helpers	24–27	<code>request</code> (called at 14, 17)
6 helpers	29–42	<code>refreshToken</code> (called at 16)

Step	Lines	Block
7 lifecycle	none	
8 clauses		

Step 2: types (1–3)

Note
<code>ApiError</code> carries <code>status</code> ; <code>super(message)</code> sets the message; fine; <code>catch (e)</code> callers need <code>instanceof ApiError</code>

Step 3: state (6–7)

Line	Note
6	<code>access</code> is mutable per instance; written at 16
7	<code>refreshing</code> is the in-flight refresh Promise or <code>null</code> : this is the clause-3 mechanism; shared across concurrent <code>get</code> calls

Step 4: signatures

Line	Note
13	<code>get</code> async, generic <code>T</code> , returns <code>Promise<T></code>
24	<code>request</code> is NOT async but returns <code>fetch(...)</code> : a Promise by pass-through; every caller must <code>await</code>
29	<code>refreshToken</code> async, returns <code>Promise<string></code>

Step 5: `get` (13–22)

Line	Note
14	awaited; <code>res</code> holds a Response (any status)
15	401 check first: correct order relative to line 20
16	awaits the refresh and stores the new token; yield point
17	retry awaited; <code>retry</code> holds a Response of any status
18	<code>retry.json()</code> returned un-awaited: fine, <code>get</code> is async so the Promise is flattened; BUT <code>retry.ok</code> is never checked: a second 401 or a 500 on the retry is parsed as <code>T</code> ; clause 4 violated on the retry path; also an infinite-refresh loop is correctly avoided (no recursion), say so
20	non-2xx other than 401 throws <code>ApiError</code> : clause 4 on the first path; correct
21	<code>return res.json()</code> : flattened; <code>T</code> is a claim, not a check

Step 6: `request` (24–27), called with `path`

Line	Note
25	logs <code>this.access</code> : the bearer token goes to the log; clause 5 violated
26	<code>fetch</code> returned un-awaited by a non-async function: pass-through Promise; fine, callers await; no timeout (design note)

Step 6: `refreshToken` (29–42), called at 16

Line	Note
30	if a refresh is in flight, return the SAME Promise: clause 3 mechanism; correct
31–38	an async IIFE assigned synchronously to <code>refreshing</code> BEFORE any await: so a second caller arriving during line 32's await sees <code>refreshing</code> set; correct ordering
32	awaited POST
36	<code>res.ok</code> not checked: a 401/500 from the token endpoint is parsed; <code>data.access</code> is <code>undefined</code> ; line 37 returns <code>undefined</code> as a string
37	<code>data.access</code> typed <code>any</code> ; returned as <code>string</code> by claim
39	awaits its own Promise
40	sets <code>refreshing = null</code> after success; if the IIFE REJECTS, line 39 throws and line 40 never runs: <code>refreshing</code> stays set to a rejected Promise, and every future call at line 30 returns that same rejected Promise forever: the client is permanently broken after one failed refresh
41	returns the token

Step 8: clauses

Clause	Line	Status
1 authenticated GET, parsed JSON	14–21	implemented
2 refresh once, retry once	15–18	implemented; retry response unchecked
3 single shared refresh	30–31	implemented
4 throw <code>ApiError</code> on non-2xx	20	first path yes; retry path (18) no; refresh endpoint (36) no
5 never log tokens	25	violated

Findings

1. Line 25: `console.log(..., this.access)` logs the bearer token; remove.
2. Lines 39–40: if the refresh rejects, `refreshing` is never reset; every later `get` reuses the rejected Promise; use `try { ... } finally { this.refreshing = null }`.
3. Line 18: retry response not checked with `res.ok`; a second 401 or any 5xx is parsed and returned as `T`; apply the same `if (!retry.ok) throw new ApiError(...)`.
4. Line 36: token endpoint response not checked; `data.access` may be `undefined`, which is then stored as the access token and sent as `"Bearer undefined"` on every later call.
5. Line 26: no timeout on `fetch` (design note).
6. Line 37: `data.access` is `any`; validate it is a non-empty string.

Positives: the in-flight Promise pattern on 30–31 correctly dedupes concurrent refreshes and is assigned before the first `await`; 401 is checked before `!res.ok`; no recursive retry; `ApiError` carries the status.

Confidence 2 • Request changes.

Clause 3 is done well; clause 5 is a security finding; clause 4 fails on two of three paths; the stuck-rejection bug breaks the client after one bad refresh.

Program D — Session store (the clean one)

Requirement

1. `create(userId)` returns a new session id; `get(id)` returns the session or `null`; `touch(id)` extends expiry; `destroy(id)` removes.
2. Sessions expire `ttlMs` after last touch; expired sessions are removed by a sweep every `sweepMs` and are never returned by `get`.
3. Returned session objects must be copies: callers cannot mutate the store.
4. `close()` stops the sweep so the process can exit.
5. `size()` reports live sessions.

```

1  import { randomUUID } from "crypto";
2
3  export interface Session { id: string; userId: string; createdAt: number; expiresAt: number }
4
5  export class SessionStore {
6      private sessions = new Map<string, Session>();
7      private sweep: ReturnType<typeof setInterval> | null = null;
8
9      constructor(private ttlMs: number, sweepMs: number, private now: () => number = Date.now) {
10         if (!Number.isFinite(ttlMs) || ttlMs <= 0) throw new Error("ttlMs must be > 0");
11         this.sweep = setInterval(() => this.evict(), sweepMs);
12         this.sweep.unref?.();
13     }
14
15     create(userId: string): string {
16         if (typeof userId !== "string" || userId.length === 0) throw new Error("userId
required");
17         const id = randomUUID();
18         const t = this.now();
19         this.sessions.set(id, { id, userId, createdAt: t, expiresAt: t + this.ttlMs });
20         return id;
21     }
22
23     get(id: string): Session | null {
24         const s = this.sessions.get(id);
25         if (!s) return null;
26         if (s.expiresAt <= this.now()) {
27             this.sessions.delete(id);
28             return null;
29         }
30         return { ...s };
31     }
32
33     touch(id: string): boolean {
34         const s = this.sessions.get(id);
35         if (!s || s.expiresAt <= this.now()) return false;
36         s.expiresAt = this.now() + this.ttlMs;
37         return true;
38     }
39
40     destroy(id: string): boolean {
41         return this.sessions.delete(id);
42     }
43
44     size(): number {
45         this.evict();
46         return this.sessions.size;
47     }
48
49     close(): void {
50         if (this.sweep !== null) { clearInterval(this.sweep); this.sweep = null; }
51     }
52
53     private evict(): void {
54         const t = this.now();
55         for (const [id, s] of this.sessions) {
56             if (s.expiresAt <= t) this.sessions.delete(id);
57         }
58     }
59 }

```

Reading sequence for D

Step	Lines	Block
2 types	3	Session
3 state	6–7	sessions, sweep
4 signatures	9, 15, 23, 33, 40, 44, 49	
5 bodies	15–51	public methods in call order: create, get, touch, destroy, size, close

Step	Lines	Block
6 helpers	53–58	<code>evict</code> (called by the interval, and by <code>size</code>)
7 lifecycle	11–12, 49–51	interval creation, <code>unref</code> , <code>close</code>
8 clauses		

Notes, in sequence

Line	Note
3	all four fields required; no <code>?</code> ; <code>expiresAt</code> is a number
6	<code>Map</code> keyed by UUID: no prototype issue; ids are not user-controlled
7	<code>sweep</code> nullable; checked with <code>!== null</code>
9	<code>now</code> is injectable; testable; defaults to <code>Date.now</code>
10	validates <code>ttlMs</code> ; <code>sweepMs</code> is not validated (nit: <code>setInterval(fn, NaN)</code> behaves as 0 ms)
11	interval stored so it can be cleared
12	<code>unref?.()</code> so the process can exit even without <code>close</code> ; optional call because browsers lack <code>unref</code>
16	validates <code>userId</code>
17	<code>randomUUID</code> is sync; fine
19	stores a NEW object; <code>Map.set</code>
24	<code>s</code> holds the stored object or <code>undefined</code>
25	<code>!s</code> is correct here: <code>s</code> is an object or <code>undefined</code> , never <code>0/""</code>
26	boundary: <code><=</code> means a session expiring exactly now is expired; consistent with 35 and 56
27	lazy eviction on read; fine
30	returns <code>{ ...s }</code> : a shallow copy; all fields are primitives, so this is a full copy; clause 3 satisfied
34–36	<code>touch</code> mutates the STORED object (intended: that is how expiry is extended); the caller holds no alias because <code>get</code> returned a copy
41	<code>delete</code> returns boolean: matches the signature
45	<code>size()</code> evicts first so the count is live; clause 5
50	clears and nulls; clause 4
55–56	deleting from a <code>Map</code> while iterating it is safe in JS; evicts by the same <code><=</code> rule

Clauses

Clause	Line	Status
1 create/get/touch/destroy	15–42	implemented
2 expiry and sweep, never returned	26–28, 53–58	implemented, both lazy and swept
3 copies returned	30	implemented
4 close stops sweep	50, plus <code>unref</code> at 12	implemented
5 size reports live	45	implemented

Findings

- Nit: `sweepMs` not validated at 9–10; `NaN` or `0` would spin the interval.
- Nit: `touch` returns `false` for expired but does not delete the expired entry (the sweep will); harmless.

Positives: injectable clock; stored interval with `unref` and `close`; `Map` with non-user keys; copy on `get` with all-primitive fields so shallow is complete; consistent `<=` boundary in three places; `!s` used only where `s` is an object; lazy eviction plus sweep; input validation on both constructor and `create`.

Confidence 5 · Approve.

If you found more than the two nits, name the Q that produced each extra and check it against the clauses: it will not point at one.

Scoring yourself

Program	Must-find	Confidence / verdict
A	auth bypass (39 vs 36), missing await (45), row mutation (23), float cents (24)	1 · Reject
B	<code>pop</code> LIFO (34), untracked retry timer vs <code>stop</code> (47/24), stats gap (28/52)	2 · Request changes
C	token logged (25), <code>refreshing</code> stuck on rejection (39–40), retry <code>ok</code> unchecked (18)	2 · Request changes
D	nothing beyond two nits	5 · Approve

For every miss, say the line and the Q, once. For every invented finding on D, say which clause it would have violated, and notice that it does not.